P2079R8
Parallel scheduler

Published Proposal, 2025-05-18

This version:
http://wg21.link/P2079R8

Authors:
Lucian Radu Teodorescu (Garmin)
Ruslan Arutyunyan (Intel)
Lee Howes
Michael Voss (Intel)

Audience:
LWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

A standard execution context based on the facilities in [P2300R10] that implements parallel-forward-progress to maximize portability. A set of parallel schedulers have an underlying shared thread pool implementation, and may provide an interface to an OS-provided system thread pool.

Table of Contents

1. Changes

1.1. R8

- Apply feedback from LEWG meeting in Hagenberg
- Add diagram for the `bulk_chunked` and `bulk_unchunked` cases.
- Improve wording

1.2. R7

- Change the title from "System execution context" to "Parallel scheduler"
- Incorporate feedback from SG1 Hagenberg, Austria, February 2025 (rename `system_scheduler` to `parallel_scheduler`, drop run-time replaceability)
- Tweaking the replaceability API to be consistent with [P3481R1], now at R2 (support `bulk_chunked` and `bulk_unchunked`)

1.3. R6

- Incorporate feedback from SG1 and LEWG given in Wrocław, Poland, November 2024.
- Remove the wording about delegatee schedulers

- `system_scheduler`'s sender can complete with `set_error`, with a `std::exception_ptr` argument.
- Remove `system_scheduler query(get_completion_scheduler_t)` overload for `set_stopped_t`.
- Add `noexcept` to move, copy constructor, move/copy assignment operators (special member functions) of `system_scheduler`.
- Add lvalue `connect` to to system sender class.
- Add `sender_concept`, `completion_signatures` and `get_env` to the definition of the system sender class, making it a sender.
- Mandate replaceability
- Define replaceability mechanism (both link-time and runtime)
- Define replaceability API
- Add Specification section

§ 1.4. R5

- Streamline the paper
- Make replaceability availability implementation-defined
- Make replaceability API implementation-defined
- Replace the use of `system_context` class with direct calls to `get_system_scheduler()`
- Update user-facing API
- Relax the lifetime guarantees to allow using system scheduler outside of `main()`

§ 1.5. R4

- Add more design considerations & goals.
- Add comparison of different replaceability options
- Add motivation for replaceability ABI standardization
- Add the example of the ABI for replacement
- Strengthen the lifetime guarantees.

1.6. R3

- Remove `execute_all` and `execute_chunk`. Replace with compile-time customization and a design discussion.
- Add design discussion about the approach we should take for customization and the extent to which the context should be implementation-defined.
- Add design discussion for an explicit `system_context` class.
- Add design discussion about priorities.

1.7. R2

- Significant redesign to fit in [P2300R10] model.
- Strictly limit to parallel progress without control over the level of parallelism.
- Remove direct support for task groups, delegating that to `async_scope`.

1.8. R1

- Minor modifications

1.9. R0

- First revision

2. Introduction

[P2300R10] describes a rounded set of primitives for asynchronous and parallel execution that give a firm grounding for the future. However, the paper lacks a standard execution context and scheduler. It has been broadly accepted that we need some sort of standard scheduler.

As part of [P3109R0], system context was voted as a must-have for the initial release of senders/receivers. It provides a convenient and scalable way of spawning concurrent work for the users of senders/receivers.

As noted in [P2079R1], an earlier revision of this paper, the `static_thread_pool` included in later revisions of [P0443R14] had many shortcomings. This was removed from [P2300R10] based on that

and other input.

One of the biggest problems with local thread pools is that they lead to CPU oversubscription. This introduces a performance problem for complex systems that are composed from many independent parts.

Another problem that system context is aiming to solve is the composability of components that may rely on different parallel engines. An application might have multiple parts, possibly in different binaries; different parts of the application may not know of each other. Thus, different parts of the application might use different parallel engines. This can create several problems:

- oversubscription because of different thread pools
- problems with nested parallel loops (one parallel loop is called from the other)
- problems related to interaction between different parallel engines
- etc.

To solve these problems we propose a parallel execution context that:

- can be shared between multiple parts of the application
- does not suffer from oversubscription
- can integrate with the OS scheduler
- can be replaced by the user to compose well with other parallel runtimes

This parallel execution context is called in this paper the **system context**. The users can obtain a scheduler from this system context.

Note: SG1 has expressed a desire to use the name "parallel scheduler" instead of "system execution scheduler", so that we can use "system execution scheduler" for a concurrent-forward-progress execution context in the future. For simplicity reasons, the paper uses the term "system context" to refer to the context of the parallel scheduler.

§ 2.1. Design overview

The system context is a parallel execution context of undefined size, supporting explicitly *parallel forward progress*.

The execution resources of the system context are envisioned to be shared across all binaries in the same process. System scheduler works best with CPU-intensive workloads, and thus, limiting oversubscription is a key goal.

By default, the system context should be able to use the OS scheduler, if the OS has one. On systems where the OS scheduler is not available, the system context will have a generic implementation that acts like a thread pool.

For enabling the users to hand-tune the performance of their applications, and for fulfilling the composability requirements, the system context should be replaceable. The user should be able to replace the default implementation of the system context with a custom one that fits their needs.

Other key concerns of this design are:

- Extensibility: being able to extend the design to work with new additions to the senders/receivers framework.
- Lifetime: as system context is a global resource, we need to pay attention to the lifetime of this resource.
- Performance: as we envision this to be used in many cases to spawn concurrent work, performance considerations are important.

§ 3. Examples

As a simple parallel scheduler we can use it locally, and `sync_wait` on the work to make sure that it is complete. With forward progress delegation this would also allow the scheduler to delegate work to the blocked thread. This example is derived from the Hello World example in [P2300R10]. Note that it only adds a well-defined context object, and queries that for the scheduler. Everything else is unchanged about the example.

```
using namespace = std::execution;

scheduler auto sch = get_parallel_scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = std::this_thread::sync_wait(add_42).value();
```

We can structure the same thing using `on`, which better matches structured concurrency:

```
using namespace std::execution;

scheduler auto sch = get_parallel_scheduler();

sender auto hi = then(just(), []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = std::this_thread::sync_wait(on(sch, add_42)).value();
```

The parallel scheduler customizes `bulk`, so we can use `bulk` dependent on the scheduler. Here we use it in structured form using the parameterless `get_scheduler` that retrieves the scheduler from the receiver, combined with `on`:

```
using namespace std::execution;

auto bar() {
  return
    let_value(
      read_env(get_scheduler),      // Fetch scheduler from receiver.
      [](auto current_sched) {
        return bulk(
          current_sched.schedule(),
          1,                        // Only 1 bulk task as a lazy way of ma
          [](auto idx){
            std::cout << "Index: " << idx << "\n";
          })
      });
}

void foo()
{
  auto [i] = std::this_thread::sync_wait(
    on(
      get_parallel_scheduler(),       // Start bar on the system's parall
```

```
      bar()))                          // and propagate it through the rece
    .value();
}
```

Use `async_scope` and a custom system context implementation linked in to the process (through a mechanism undefined in the example). This might be how a given platform exposes a custom context. In this case we assume it has no threads of its own and has to take over the main thread through an custom `drive()` operation that can be looped until a callback requests `exit` on the context.

```
using namespace std::execution;

int result = 0;

{
  async_scope scope;
  scheduler auto sch = get_parallel_scheduler();

  sender auto work =
    then(just(), [&](auto sched) {

      int val = 13;

      auto print_sender = then(just(), [val]{
        std::cout << "Hello world! Have an int with value: " << val << "\n"
      });

      // spawn the print_sender on sched to make sure it
      // completes before shutdown
      scope.spawn(on(sch, std::move(print_sender)));

      return val;
    });

  scope.spawn(on(sch, std::move(work)));

  // This is custom code for a single-threaded context that we have replace
  // We need to drive it in main.
  // It is not directly sender-aware, like any pre-existing work loop, but
  // does provide an exit operation. We may call this from a callback chain
  // after the scope becomes empty.
  // We use a temporary terminal_scope here to separate the shut down
```

```
  // operation and block for it at the end of main, knowing it will complet
  async_scope terminal_scope;
  terminal_scope.spawn(
    scope.on_empty() | then([](my_os::exit(sch))));
  my_os::drive(sch);
  std::this_thread::sync_wait(terminal_scope);
};

// The scope ensured that all work is safely joined, so result contains 13
std::cout << "Result: " << result << "\n";

// and destruction of the context is now safe
```

✧ ✧ ✧

To change the implementation of system scheduler at link-time, one might do it the following way (very simplistic example):

```
namespace std::execution::system_context_replaceability {
  extern __attribute__((__weak__))
  std::shared_ptr<parallel_scheduler> query_parallel_scheduler_backend() {
    return std::make_shared<my_parallel_scheduler_impl>();
  }
}
```

4. Design

4.1. User facing API

```
parallel_scheduler get_parallel_scheduler();

class parallel_scheduler { // exposition only
public:
  parallel_scheduler() = delete;
  ~parallel_scheduler();

  parallel_scheduler(const parallel_scheduler&) noexcept;
```

```
  parallel_scheduler(parallel_scheduler&&) noexcept;
  parallel_scheduler& operator=(const parallel_scheduler&) noexcept;
  parallel_scheduler& operator=(parallel_scheduler&&) noexcept;

  bool operator==(const parallel_scheduler&) const noexcept;

  forward_progress_guarantee query(get_forward_progress_guarantee_t) const

  unspecified-parallel-sender schedule() const noexcept;
  // customization for bulk
};

class unspecified-parallel-sender { // exposition only
public:
  using sender_concept = sender_t;
  using completion_signatures = execution::completion_signatures<
    set_value_t(),
    set_stopped_t(),
    set_error_t(exception_ptr)>;

  impl-defined-environment get_env() const noexcept;

  parallel_scheduler query(get_completion_scheduler_t<set_value_t>) const n

  template<receiver R>
  requires receiver_of<R>
  impl-defined-operation_state connect(R&&) &
      noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<R>, R>);
  template<receiver R>
  requires receiver_of<R>
  impl-defined-operation_state connect(R&&) &&
      noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<R>, R>);
};
```

- `get_parallel_scheduler()` returns a scheduler that provides a view on some underlying execution context supporting *parallel forward progress*, with at least one thread of execution (which may be the main thread).
- two objects returned by `get_parallel_scheduler()` most frequently share the same execution context. If work submitted by one can consume the underlying thread pool, that can block progress of another. One notable exception is when the backend scheduler needs to be re-created.

- if `Sch` is the type of object returned by `get_parallel_scheduler()`, then:
  - `Sch` is implementation-defined, but must be nameable.
  - `Sch` models the `scheduler` concept.
  - `Sch` implements the `get_forward_progress_guarantee` query to return `parallel`.
  - `Sch` implements `schedule` customization point to return an implementation-defined `sender` type.
  - `schedule` calls on `Sch` are non-blocking operations.
  - `Sch` implements the `bulk` CPO to customize the `bulk` sender adapter such that:
    - when `execution::set_value(r, args...)` is called on the created `receiver`, an agent is created with parallel forward progress on the underlying system context for each `i` of type `Shape` from `0` to `sh`, where `sh` is the shape parameter to the `bulk` call, that calls `f(i, args...)`.
- if `sch` is an object returned by `get_parallel_scheduler()`, then:
  - the lifetime of `sch` does not have to outlive work submitted to it.
  - `sch` is both move and copy constructible and assignable.
  - if `sch2` is another object returned by `get_parallel_scheduler()`, then `sch == sch2` is `true` if and only if they share the same backend implementation.
- if `snd` is a sender obtaining from calling `schedule` on the scheduler returned by `get_parallel_scheduler()`, and `Snd` its type, then:
  - `Snd` is implementation-defined, but must be nameable.
  - `Snd` models the `sender` concept.
  - `Snd` exposes an environment that implements the `get_completion_scheduler` query for the value completion channel, returning an object that equals to itself.
  - `connect`ing `snd` to a `receiver` object and calling `start()` on the resulting operation state are non-blocking operations.
  - if `snd` is connected with a `receiver` that supports the `get_stop_token` query and if that `stop_token` is stopped, operations on which `start` has been called, but are not yet running (and are hence not yet guaranteed to make progress) **must** complete with `set_stopped` as soon as is practical.
  - if `snd` is connected with a `receiver`, and the resulting operation is started but the implementation reaches an error condition, the operation must complete with

`set_error(ep)`, where `ep` is of type `std::exception_ptr`.

- The `bulk` algorithm is customized for the scheduler returned by `get_parallel_scheduler()`
  - the corresponding sender has the same properties as the sender returned by `schedule()`;
  - the functor given to `bulk` is invoked from threads belonging to the system execution context.

§ 4.2. Replaceability API

```
namespace std::execution::system_context_replaceability {
  struct parallel_scheduler;

  // Called by the frontend.
  // Users might replace this function.
  shared_ptr<parallel_scheduler> query_parallel_scheduler_backend();

  // Implemented by the frontend.
  struct receiver {
    virtual ~receiver() = default;

  protected:
    // exposition only
    virtual bool query-env(property-id, void*) noexcept = 0;

  public:

    receiver(const receiver&) = delete;
    receiver(receiver&&) = delete;
    receiver& operator=(const receiver&) = delete;
    receiver& operator=(receiver&&) = delete;

    virtual void set_value() noexcept = 0;
    virtual void set_error(std::exception_ptr) noexcept = 0;
    virtual void set_stopped() noexcept = 0;

    template <class-type P> // class-type is defined in [execution.syn]
    std::optional<P> try_query() noexcept;
  };
```

```
  // Implemented by the frontend.
  struct bulk_item_receiver : receiver {
    virtual void execute(uint32_t begin, uint32_t end) noexcept = 0;
  };

  // Implemented by the backend.
  struct parallel_scheduler_backend {
    virtual ~parallel_scheduler_backend() = default;

    virtual void schedule(receiver&, std::span<std::byte>) noexcept = 0;
    virtual void schedule_bulk_chunked(uint32_t, bulk_item_receiver&, std::
    virtual void schedule_bulk_unchunked(uint32_t, bulk_item_receiver&, std
  };
}
```

- > NOTE: for the current exposition we call *backend* the part of the application that implements a (possibly custom) system context, and *frontend* the part of the application around `execution::parallel_scheduler` that uses that (possibly custom) system context. The *frontend* part consists of library code that implements the user-facing API to call the interfaces implemented by the backend.
- `query_parallel_scheduler_backend` returns a non-null shared pointer to an object that implements the `parallel_scheduler_backend` interface.
  - > NOTE: it is expected for users to want to have link-time replacements of this function.
- > NOTE: `receiver` and `bulk_item_receiver` are interfaces that are implemented by the frontend side; only `parallel_scheduler_backend` is expected to be implemented by a system context implementation.
- `receiver` class provides the backend a way to query environment properties from the frontend receiver object.
  - if, on the frontend side, the sender obtained from a system scheduler is connected to a receiver that has an environment for which `get_stop_token` returns an `inplace_stop_token` object, then `receiver::try_query<inplace_stop_token>()` returns an optional of a value equal with the stop token from the environment;

- > NOTE: depending on the implementation of the frontend, not all the environment properties may be available to the backend.

- if `sch` is an object that implements `parallel_scheduler_backend` interface (and can be returned via `query_parallel_scheduler_backend`), then the following must be true:
  - if `sch.schedule()` is called passing `r` of type `receiver*` and `s` of type `std::span<std::byte>`, then:
  - at least one of `set_value`, `set_error`, or `set_stopped` must be eventually called on `r`;
    - `set_value` is called to signal a successful scheduling of the work; it must be called on a thread belonging to system execution context;
    - if `set_value` cannot be called, then `set_error` must be called to signal the scheduling error;
    - `set_stopped` may be called on `r` to signal that the execution of work is no longer needed;
  - the `std::span<std::byte>` object `s` represents a memory region that can be used by the backend to store data needed for the scheduling operation.
  - > NOTE: if the receiver `r` exposes a property for a stop token, and stop was requested on that stop token, then `set_stopped` may be called, but this is not guaranteed.

- if `sch.schedule_bulk_chunked()` is called passing `n` of type `uint32_t`, `r` of type `bulk_item_receiver*` and `s` of type `std::span<std::byte>`, then:
  - at least one of `set_value`, `set_error`, or `set_stopped` must be eventually called on `r`;
    - `set_value` is called to signal a successful scheduling of the work; it must be called on a thread belonging to system execution context;
    - if `set_value` cannot be called, then `set_error` must be called to signal the scheduling error;
    - `set_stopped` may be called on `r` to signal that the execution of work is no longer needed;
  - the `std::span<std::byte>` object `s` represents a memory region that can be used by the backend to store data needed for the scheduling operation.
  - > NOTE: if the receiver `r` exposes a property for a stop token, and stop was requested on that stop token, then `set_stopped` may be called, but this is not guaranteed.

- if `set_value` is called on `r`, then `start` must be called on `r` `n` times, where `n` is the value passed to `schedule_bulk_chunked`.
  - the `start` method is called on `r` with non-overlapping intervals [`begin`, `end`) to signal the start of the work for the elements in range [`begin`, `end`), where `0` <= `begin` < `end` <= `n`.
  - the `start` method must be called on a thread belonging to the system execution context.
  - the `start` method must be called before `set_value()` method is called.
- `schedule_bulk_unchunked` does the same thing as `schedule_bulk_chunked`, but the calls to `start` are using parameters `(i, i+1)` for `i` in `[0, n)`.
- if in the process of calling `start` on `r`, the implementation detects that the work cannot be started, then `set_error` must be called on `r` to signal the error; in this case `r` may not get all the expected `n` calls to `start`.
- if in the process of calling `start` on `r`, the scheduling process is cancelled, then `set_stopped` must be called on `r` to signal the cancellation; in this case `r` may not get all the expected `n` calls to `start`.

§ 5. Design discussion and decisions

§ 5.1. To drive or not to drive

On single-threaded systems (e.g., freestanding implementations) or on systems in which the main thread has special significance (e.g., to run the Qt main loop), it's important to allow scheduling work on the main thread. For this, we need the main thread to *drive* work execution.

The earlier version of this paper, [P2079R2], included `execute_all` and `execute_chunk` operations to integrate with senders. In this version we have removed them because they imply certain requirements of forward progress delegation on the system context and it is not clear whether or not they should be called. We envision a separate paper that adds the support for drive-ability, which is decoupled by this paper.

We can simplify this discussion to a single function:

```
void drive(system_context& ctx, sender auto snd);
```

Let's assume we have a single-threaded environment, and a means of customizing the system context for this environment. We know we need a way to donate `main`'s thread to this context, it is the only thread we have available. Assuming that we want a `drive` operation in some form, our choices are to:

- define our `drive` operation, so that it is standard, and we use it on this system.
- or allow the customization to define a custom `drive` operation related to the specific single-threaded environment.

With a standard `drive` of this sort (or of the more complex design in [P2079R2]) we might write an example to use it directly:

```
system_context ctx;
auto snd = on(ctx, doWork());
drive(ctx, std::move(snd));
```

Without drive, we rely on an `async_scope` to spawn the work and some system-specific drive operation:

```
system_context ctx;
async_scope scope;
auto snd = on(ctx, doWork());
scope.spawn(std::move(snd));
custom_drive_operation(ctx);
```

Neither of the two variants is very portable. The first variant requires applications that don't care about drive-ability to call `drive`, while the second variant requires custom pluming to tie the main thread with the system scheduler.

We envision a new paper that adds support for a *main scheduler* similar to the *system scheduler*. The main scheduler, for hosted implementations would be typically different than the system scheduler. On the other hand, on freestanding implementations, the main scheduler and system scheduler can share the same underlying implementation, and both of them can execute work on the main thread; in this mode, the main scheduler is required to be driven, so that system scheduler can execute work.

Keeping those two topic as separate papers allows to make progress independently.

§ 5.2. Freestanding implementations

This paper payed attention to freestanding implementations, but doesn't make any wording proposals for them. We express a strong desire for the system scheduler to work on freestanding implementations, but leave the details to a different paper.

We envision that, a followup specification will ensure that the system scheduler will work in freestanding implementations by sharing the implementation with the main scheduler, which is driven by the main thread.

§ 5.3. Making system context replaceable

The system context aims to allow people to implement an application that is dependent only on parallel forward progress and to port it to a wide range of systems. As long as an application does not rely on concurrency, and restricts itself to only the system context, we should be able to scale from single threaded systems to highly parallel systems.

In the extreme, this might mean porting to an embedded system with a very specific idea of an execution context. Such a system might not have a multi-threading support at all, and thus the system context not only runs with single thread, but actually runs on the system's only thread. We might build the context on top of a UI thread, or we might want to swap out the system-provided implementation with one from a vendor (like Intel) with experience writing optimized threading runtimes.

The latter is also important for the composability of the existing code with the system context, i.e., if Intel Threading building blocks (oneTBB) is used by somebody and they want to start using the system context as well, it's likely that the users want to replace system context implementation with oneTBB because in that case they would have one thread pool and work scheduler underneath.

We should allow customization of the system context to cover this full range of cases.

To achieve this we see options:

1. Link-time replaceability. This could be achieved using weak symbols, or by choosing a runtime library to pull in using build options.
2. Run-time replaceability. This could be achieved by subclassing and requiring certain calls to be made early in the process.
3. Compile-time replaceability. This could be achieved by importing different headers, by macro definitions on the command line or various other mechanisms.

Link-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to replacing `operator new`.
- Pro: more predictable, in that it can be guaranteed to be application-global.
- Pro: some of the type erasure and indirection can be removed in practice with link-time optimization.
- Con: it requires defining the ABI and thus, in some cases, would require some type erasure and some inefficiency.
- Con: harder to get it correctly with shared libraries (e.g., DLLs might have different replaced versions of the system scheduler).
- Con: the replacement might depend on the order of linking.

Run-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to `std::set_terminate()`.
- Pro: easier to achieve consistent behavior on applications with shared libraries (e.g., Windows has the same version of C++ standard library in DLL).
- Pro: a program can have multiple implementations of system scheduler.
- Con: race conditions between replacing the system scheduler and using it to spawn work (for buggy implementations).
- Con: race conditions between using the scheduler and replacing it, which makes the program hard to follow.
- Con: implies going over an ABI, and cannot be optimized at link-time.
- Con: different implementation may allocate resources for the system scheduler at startup, and then, at the start of main, the implementation is replaced (this is mainly a QOI issue).

Compile-time replaceability has the following characteristics:

- Pro: users can do this with a type-def that can be used everywhere and switched.
- Con: potential problems with ODR violations.
- Con: doesn’t support shareability across different binaries of the same process

The paper considers compile-time replaceability as not being a viable option because it easily breaks one of the fundamental design principles of a system context, i.e. having one, shared, application-wide execution context, which avoids oversubscription.

After discussing run-time replaceability in SG1, in Hagenberg, we decided to also drop run-time replaceability. Thus, the only remaining option is link-time replaceability.

Replaceability is also part of the [P2900R8] proposal for the contract-violation handler. The paper proposes that whether the handler is replaceable to be implementation-defined. If an implementation chooses to support replaceability, it shall be done similar to replacing the global `operator new` and `operator delete` (link-time replaceability).

The replaceability topic is highly controversial. Some people think that link-time replaceability is the way to go; Adobe supports this position. Others think that run-time replaceability is the way to go; Bloomberg supports this latter position.

The feedback we received from Microsoft, is that they will likely not support replaceability on their platforms. They would prefer that we offer implementations an option to not implement replaceability. Moreover, for systems where replaceability is supported they would prefer to make the replaceability mechanism to be implementation defined.

The authors disagree with the idea that replaceability is not needed for Windows platforms (or other platforms that provide an OS scheduler). The OS scheduler is optimized for certain workloads, and it's not the best choice for all workloads. This way not providing replaceability options have the following drawbacks:

- it limits the ability to hand-tune the performance of the application (when system scheduler is used);
- it limits the `parallel_scheduler` ability to be used in for CPU-intensive workloads;
- it limits the `parallel_scheduler` ability to be used for platforms with accelerators;
- it limits the composability of the system context with other parallel runtimes (while avoiding oversubscription).

The poll taken in Wrocław, Poland, November 2024, showed that the majority of the participants support mandating replaceability, as well as specifying the mechanism of replaceability and the API for replaceability.

In accordance with the feedback, and the beliefs of the authors, the paper proposes the following:

- mandate replaceability
- make the replaceability mechanism to be link-time
- define a replaceability API

§ 5.4. Replaceability details

To replace the system scheduler, the user needs to do the following:

- Implement a system scheduler behind the `std::system_context_replaceability::parallel_scheduler_backend` interface.
- Follow the implementation instructions to replace the `std::system_context_replaceability::query_parallel_scheduler_backend` symbol, with a function that returns the desired backend implementation.

If parallel scheduler backend is replaced, the entire program will only see the replacement, and not the default implementation. However, the parallel scheduler backend object might be destroyed before other objects that depend on it. In this case, the link-time replaced function is called again to create a new new parallel scheduler backend. Thus, multiple parallel scheduler backends can be created, but only one of them is active at a time.

§ 5.5. Extensibility

The `std::execution` framework is expected to grow over time. We expect to add time-based scheduling, async I/O, priority-based scheduling, and other for now unforeseen functionality. The system context framework needs to be designed in such a way that it allows for extensibility.

Whatever the replaceability mechanism is, we need to ensure that new features can be added to the system context in a backwards-compatible manner.

There are two levels in which we can extend the system context:

1. Add more types of schedulers, beside the system scheduler.
2. Add more features to the existing scheduler.

The first type of extensibility can easily be solved by adding new getters for the new types of schedulers. Different types of schedulers should be able to be replaced separately; e.g., one should be able to replace the I/O scheduler without replacing the system scheduler. The discussed replaceability mechanisms support this.

To extend existing schedulers, one can pass properties to it, via the receiver. For example, adding priorities to the system scheduler can be done by adding a priority property to the type-erased receiver object. The list of properties that can be passed to the backend is not finite; however both the frontend and the backend must have knowledge about the supported properties.

5.6. Shareability

One of the motivations of this paper is to stop the proliferation of local thread pools, which can lead to CPU oversubscription. If multiple binaries are used in the same process, we don't want each binary to have its own implementation of system context. Instead, we would want to share the same underlying implementation.

The paper mandates shareability, but leaves the details of shareability to be implementation-defined (they are different for each backend).

5.7. Performance

To support shareability and replaceability, system context calls may need to go across binary boundaries, over the defined API. A common approach for this is to have COM-like objects. However, the problem with that approach is that it requires memory allocation, which might be a costly operation. This becomes problematic if we aim to encourage programmers to use the system context for spawning work in a concurrent system.

While there are some costs associated with implementing all the goals stated here, we want the implementation of the system context to be as efficient as possible. For example, a good implementation should avoid memory allocation for the common case in which the default implementation is utilized for a platform.

This paper cannot recommend the specific implementation techniques that should be used to maximize performance; these are considered Quality of Implementation (QOI) details.

5.8. Lifetime

Underneath the system scheduler, there is a singleton of some sort. We need to specify the lifetime of this object and everything that derives from it.

Revision R4 of the paper mandates that the lifetime of any `system_context` must be fully contained within the lifetime of `main()`. The reasoning behind this is that ensuring proper construction and destruction order of static objects is typically difficult in practice. This is especially challenging during the destruction of static objects; and, by symmetry, we also did not want to guarantee the lifetime of the system scheduler before `main()`. We argued that if we took a stricter approach, we could always relax it later.

We received feedback that this was too strict. First, there are many applications where the C++ part does not have a `main()` function. Secondly, this can be considered a quality of implementation issue; implementations can always use a Phoenix singleton pattern to ensure that the underlying system context object remains alive for the duration of the entire program.

R5 revision of the paper relaxes the lifetime requirements of the system scheduler. The system scheduler can now be used in any part of a C++ program.

§ 5.9. Need for the `system_context` class

Our goal is to expose a global shared context to avoid oversubscription of threads in the system and to efficiently share a system thread pool. Underneath the `system_context` there is a singleton of some sort, potentially owned by the OS.

The question is how we expose the singleton. We have a few obvious options:

- Explicit context objects, as we've described in R2, R3 and R4 of this paper, where a `system_context` is constructed as any other context might be, and refers to a singleton underneath.
- A global `get_system_context()` function that obtains a `system_context` object, or a reference to one, representing the singleton explicitly.
- A global `get_parallel_scheduler()` function that obtains a scheduler from some singleton system context, but does not explicitly expose the context.

In R4 and earlier revisions, we opted for an explicit context object. The reasoning was that providing explicit contexts makes it easier to understand the lifetime of the schedulers. However, adding this extra class does not affect how one would reason about the lifetime of the schedulers or the work scheduled on them. Therefore, introducing an artificial scope object becomes an unnecessary burden.

There were also arguments made for adding `system_context` so that we can later add properties to it, that don't necessarily belong to `parallel_scheduler`. However, if we would later find such properties, nothing prevents us to add `system_context` class later, and make `get_parallel_scheduler()` return `get_system_context().get_parallel_scheduler()`.

Thus, the paper simply proposes a `get_parallel_scheduler()` function that returns a the system scheduler. The system context is implementation-defined and not exposed to the user.

§ 5.10. Backend environment

For custom backend implementations, it is often necessary access properties of the environment that is connected to the sender that triggers a schedule operation. Because the backend is behind a type-erased boundary, the environment that can be passed to the backend also needs to be type-erased.

We've added the possibility to encode environment properties in the receiver object. The backend can query the receiver for environment properties by using the `try_query` template method, passing the type of the property that needs querying. If the property is available, the method returns an optional with the value of the property; otherwise it returns an empty optional.

The backend can query the receiver object for the entire lifetime of the asynchronous operation (until one of the completion signals is called).

One of the implication for using types to query properties is that the exchange of environment properties needs to use concrete types, and cannot use concepts. If, for example, the backend needs to obtain a stop token, it needs to ask for a specific stop token type (like `inplace_stop_token`), and not for a `stoppable_token` concept. As another example, if the backend needs an allocator, it needs to ask for a specific allocator type, and not use `simple-allocator` concept.

At this point, the paper requires that only `inplace_stop_token` is supported by the type-erased receiver object. Other types might be supported, but it is not required.

The authors envision that the list of supported properties will grow over time.

§ 5.11. Priorities

It's broadly accepted that we need some form of priorities to tweak the behavior of the system context. This paper does not include priorities, though early drafts of R2 did. We had different designs in flight for how to achieve priorities and decided they could be added later in either approach.

The first approach is to expand one or more of the APIs. The obvious way to do this would be to add a priority-taking version of `system_context::get_scheduler()`:

```
implementation-defined-parallel_scheduler get_scheduler();
implementation-defined-parallel_scheduler get_scheduler(priority_t priority
```

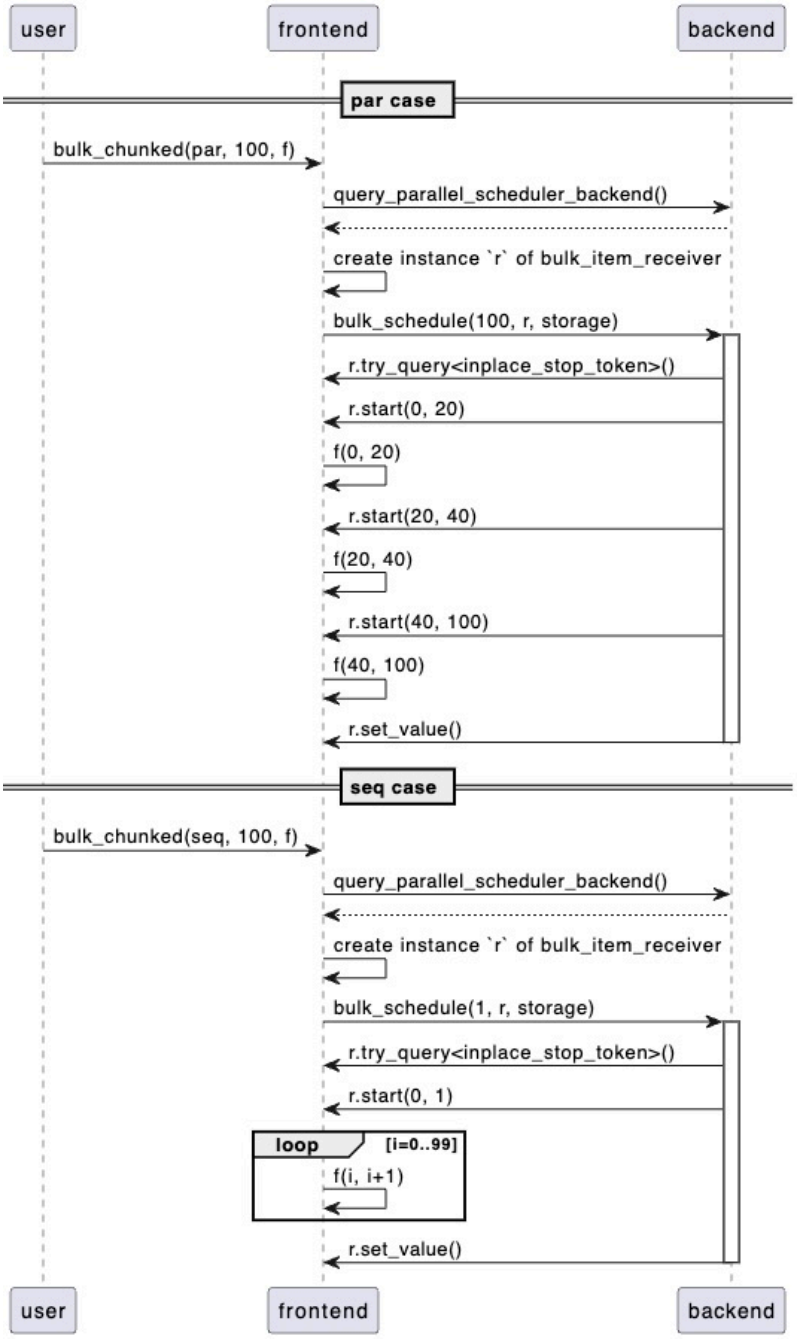
This approach would offer priorities at scheduler granularity and apply to large sections of a program at once.

The other approach, which matches the receiver query approach taken elsewhere in [P2300R10] is to add a `get_priority()` query on the receiver, which, if available, passes a priority to the scheduler in the same way that we pass an `allocator` or a `stop_token`. This would work at task granularity, for each `schedule()` call that we connect a receiver to we might pass a different priority.

In either case we can add the priority in a separate paper. It is thus not urgent that we answer this question, but we include the discussion point to explain why they were removed from the paper.

§ 5.12. Interaction between frontend and backend

The functionality of the parallel scheduler that will be used by end-users (frontend) is implemented in terms of a "backend" that can be replaced. The following diagram shows the interaction between the frontend and backend parts of the parallel scheduler.



In the first part of the image, we show how the backend will start multipel threads and call `start()` on the receiver multiple times, which will in turn, call the function given by the user.

In the second part of the image, because the `seq` execution policy was given, we call `start()` only once, and the frontend implementation will call `f` multiple times sequentially.

§ 5.13. Reference implementation

The authors prepared a reference implementation in stdexec

A few key points of the implementation:

- The implementation is divided into two parts: "frontend" and "backend". The frontend part implements the API defined in this paper and calls the backend for the actual implementation. The backend provides the actual implementation of the system context.
- Allows link-time replaceability for `system_scheduler`. Provides examples on doing this.
- (not to spec) Allows run-time replaceability for `system_scheduler`. Provides examples on doing this.
- Defines a replaceability API between the frontend and backend parts. This way, one can easily extend this interface when new features need to be added to system context.
- Uses preallocated storage on the frontend side, so that the default implementation doesn't need to allocate memory on the heap when adding new work to `system_scheduler`.
- Uses a Phoenix singleton pattern to ensure that the system scheduler is alive when needed.
- As the default implementation is created outside of the frontend part, it can be shared between multiple binaries in the same process.
- uses a `static_thread_pool`-based implementation as a default on generic platforms, and using `libdispatch` as default implementation on MacOS

5.14. Addressing received feedback

5.14.1. Allow for system context to borrow threads

Early feedback on the paper from Sean Parent suggested a need for the system context to support a configuration where it carries no threads of its own and takes over the main thread. While in [P2079R2] we proposed `execute_chunk` and `execute_all`, these enforce a particular implementation on the underlying execution context. Instead, we simplify the proposal by removing this functionality and assuming that it is implemented by link-time or run-time replacement of the context. We assume that the underlying mechanism to drive the context, should one be necessary, is implementation-defined. This allows for custom hooks into an OS thread pool, or a simple `drive()` method in main.

As we discussed previously, a separate paper is supposed to take care of the drive-ability aspect.

§ 5.14.2. Allow implementations to use Grand Central Dispatch and Windows Thread Pool

In the current form of the paper, we allow implementations to define the best choice for implementing the system context for a particular system. This includes using Grand Central Dispatch on Apple platforms and Windows Thread Pool on Windows.

In addition, we propose implementations to allow the replaceability of the system context implementation. This means that users should be allowed to write their own system context implementations that depend on OS facilities or a necessity to use some vendor (like Intel) specific solutions for parallelism.

§ 5.14.3. Priorities and elastic pools

Feedback from Sean Parent:

> There is so much in that proposal that is not specified. What requirements are placed on the system scheduler? Most system schedulers support priorities and are elastic (i.e., blocking in the system thread pool will spin up additional threads to some limit).

The lack of details in the specification is intentional, allowing implementers to make the best compromises for each platform. As different platforms have different needs, constraints, and optimization goals, the authors believe that it is in the best interest of the users to leave some of these details as Quality of Implementation (QOI) details.

§ 5.14.4. Implementation-defined may make things less portable

Some feedback gathered during discussions on this paper suggested that having many aspects of the paper to be implementation-defined would reduce the portability of the system context.

While it is true that people that would want to replace the system scheduler will have a harder time doing so, this will not affect the users of the system scheduler. They would still be able to the use system context and system scheduler without knowing the implementation details of those.

We have a precedence in the C++ standard for this approach with the global allocator.

5.14.5. Replaceability is not needed (at least on Windows)

Microsoft provided feedback that they will likely not support replaceability on their platforms. The feedback we received from Microsoft, is that they will likely not support replaceability on their platforms. They would prefer that we offer implementations an option to not implement replaceability. Moreover, for systems where replaceability is supported they would prefer to make the replaceability mechanism to be implementation defined.

The authors disagree with the idea that replaceability is not needed for Windows platforms (or other platforms that provide an OS scheduler). The OS scheduler is optimized for certain workloads, and it's not the best choice for all workloads. This way not providing replaceability options have the following drawbacks:

- it limits the ability to hand-tune the performance of the application (when system scheduler is used);
- it limits the `parallel_scheduler` ability to be used in for CPU-intensive workloads;
- it limits the `parallel_scheduler` ability to be used for platforms with accelerators;
- it limits the composability of the system context with other parallel runtimes (while avoiding oversubscription).

The poll taken in Wrocław, Poland, November 2024, showed that the majority of the participants support mandating replaceability, as well as specifying the mechanism of replaceability and the API for replaceability.

6. Specification

6.1. Header `<version>` synopsis 17.3.2 [version.syn]

To the `<version>` synopsis 17.3.2 [version.syn], add the following:

```
#define __cpp_lib_syncbuf                           201803L // also in <syn
#define __cpp_lib_parallel_scheduler                2025XXL // also in <exe
#define __cpp_lib_text_encoding                     202306L // also in <tex
```

§ 6.2. 8.2 Header `<execution>` synopsis 34.4 [execution.syn]

To the `<execution>` synopsis 34.4 [execution.syn], add the following at the end of the code block:

```
namespace std::execution {
  // [exec.parallel.scheduler]
  class parallel_scheduler { unspecified };
  parallel_scheduler get_parallel_scheduler();
}

// [exec.sysctxrepl]
namespace std::execution::system_context_replaceability {
  struct parallel_scheduler_backend;

  shared_ptr<parallel_scheduler_backend> query_parallel_scheduler_backe

  struct receiver {
    virtual ~receiver() = default;

    receiver(const receiver&) = delete;
    receiver(receiver&&) = delete;
    receiver& operator=(const receiver&) = delete;
    receiver& operator=(receiver&&) = delete;

  protected:
    // exposition only
    virtual bool query-env(property-id, void*) noexcept = 0;

  public:
    virtual void set_value() noexcept = 0;
    virtual void set_error(std::exception_ptr) noexcept = 0;
    virtual void set_stopped() noexcept = 0;

    template <class-type P>
    std::optional<P> try_query() noexcept;
  };

  struct bulk_item_receiver : receiver {
    virtual void execute(uint32_t, uint32_t) noexcept = 0;
  };

  struct parallel_scheduler_backend {
    virtual ~parallel_scheduler_backend() = default;
```

```
    virtual void schedule(receiver&, std::span<std::byte>) noexcept = 0;
    virtual void schedule_bulk_chunked(uint32_t, bulk_item_receiver&,
      std::span<std::byte>) noexcept = 0;
    virtual void schedule_bulk_unchunked(uint32_t, bulk_item_receiver&,
      std::span<std::byte>) noexcept = 0;
  };
}
```

§ 6.3. Parallel scheduler

Add the following as a new subsection at the end of 33 [exec]:

33.N Parallel scheduler [exec.parallel.scheduler]

1. `parallel_scheduler` is a class type that models the `scheduler` concept.
   - *[Note:* The underlying parallel execution context can be system provided, shared between applications, aiming to avoiding oversubscription. — *end note]*
   - *[Note:* The intended use for system scheduler is to have the parallel execution context shared between applications. — *end note]*
2. `get_parallel_scheduler` returns a scheduler object that represents a parallel execution context that is shared within the application.
3. *Returns:* An instance of the `execution::parallel_scheduler` class type.
4. *Effects:*
   - Calls `execution::system_context_replaceability::query_parallel_scheduler_backend()` to obtain an object of the `execution::system_context_replaceability::parallel_scheduler_backend` type and associates this object with the returned `execution::parallel_scheduler` object.
   - If the object returned by `execution::system_context_replaceability::query_parallel_scheduler_backend()` is `nullptr`, `terminate()` ([except.terminate]) is called.
5. *Remarks:*

- If the user does not specify a custom `parallel_scheduler_backend` object (see [exec.sysctxrepl]), a default is provided by the implementation.

6. Let `sch` be an object of type `execution::parallel_scheduler`, and let *`backend-of`* be a function such that an expression *`backend-of`*`(sch)` returns an object of the `execution::system_context_replaceability::parallel_scheduler_backend` type, associated with `sch`.
7. The expression `get_forward_progress_guarantee(sch)` returns `execution::parallel`.
8. Let `sch2` an object of the `execution::parallel_scheduler` type. The two objects `sch` and `sch2` compare equal if and only if *`backend-of`*`(sch) == `*`backend-of`*`(sch2)`.
9. Let `b` be an object returned by the *`backend-of`*`(sch)` expression, let `snd` be the object returned by calling `execution::schedule(sch)`, and let `rcvr` be a receiver. If `rcvr` is connected to `snd` and the resulting operation state is started, then `b.schedule(r, s)` is called, where `r` and `s` are objects such that:
    - `r` is an object of a type derived from `execution::system_context_replaceability::receiver` such that:
        - if `r.set_value()` is called, then `execution::set_value(std::move(rcvr))` is called;
        - if `r.set_error(e)` is called for an expression `e`, then `execution::set_error(std::move(rcvr), std::move(e))` is called;
        - if `r.set_stopped()` is called, then `execution::set_stopped(std::move(rcvr))` is called.
    - `s` is an object of type `std::span<std::byte>` that points to a possibly empty storage space; if the storage space is not empty it shall be available until one of `r.set_value()`, `r.set_error()` or `r.set_stopped()` is called.
10. A sender returned from `execution::schedule(sch)` shall provide a customized implementation of the `execution::bulk_chunked()` algorithm [exec.bulk]. If a receiver `rcvr` is connected to the sender returned by the `execution::bulk_chunked(snd, pol, shape, f)` expression and the resulting operation state is started, then `b.schedule_bulk_chunked(shape, r, s)` is called, where `r` and `s` are objects such as:
    - `r` is an object of a type derived from `execution::system_context_replaceability::bulk_item_receiver` such as:

- if `r.set_value(args...)` is called, then `execution::set_value(std::move(rcvr), std::move(args)...)` is called;
- if `r.set_error(e)` is called for an expression `e`, then `execution::set_error(std::move(rcvr), std::move(e))` is called;
- if `r.set_stopped()` is called, then `execution::set_stopped(std::move(rcvr))` is called.
- if `r.execute(i, j)` is called for indices `i` and `j`, then `f(i, j)` is called.

- `s` is an object of type `std::span<std::byte>` that points to a possibly empty storage space; if the storage space is not empty it shall be available until one of `r.set_value()`, `r.set_error()` or `r.set_stopped()` is called.
- *[Note:* Customizing the behavior of `bulk_chunked` affects the default implementation of `bulk`. — *end note]*

11. A sender returned from `execution::schedule(sch)` shall provide a customized implementation of the `execution::bulk_unchunked()` algorithm [exec.bulk]. If a receiver `rcvr` is connected to the sender returned by the `execution::bulk_unchunked(snd, shape, f)` expression and the resulting operation state is started, then `b.schedule_bulk_unchunked(shape, r, s)` is called, where `r` and `s` are objects such as:

    - `r` is an object of a type derived from `execution::system_context_replaceability::bulk_item_receiver` such as:

        - if `r.set_value(args...)` is called, then `execution::set_value(std::move(rcvr), std::move(args)...)` is called;
        - if `r.set_error(e)` is called for an expression `e`, then `execution::set_error(std::move(rcvr), std::move(e))` is called;
        - if `r.set_stopped()` is called, then `execution::set_stopped(std::move(rcvr))` is called.
        - if `r.execute(i, i + 1)` is called for index `i`, then `f(i)` is called.

    - `s` is an object of type `std::span<std::byte>` that points to a possibly empty storage space; if the storage space is not empty it shall be available until one of `r.set_value()`, `r.set_error()` or `r.set_stopped()` is called.

33.N Namespace `execution::system_context_replaceability` [exec.sysctxrepl]

1. Facilities in the `execution::system_context_replaceability` namespace allow users to replace the default implementation of parallel scheduler.

33.N.1 `shared_ptr<parallel_scheduler_backend> query_parallel_scheduler_backend();` [exec.sysctxrepl.query]

1. `query_parallel_scheduler_backend()` returns the implementation object for a parallel scheduler.
2. *Returns*: a non-null shared pointer to an object that implements the `parallel_scheduler_backend` interface.
3. *Remarks*: This function is replaceable ([dcl.fct.def.replace]).

33.N.2 Class `receiver` [exec.sysctxrepl.receiver]

1. `receiver` represents the type-erased receiver that will be notified by the implementations of `parallel_scheduler_backend` to trigger the completion operations. `bulk_item_receiver` is inherited from `receiver` and is used for `bulk_chunked`/`bulk_unchunked` customizations that will also receive notifications from implementations of `parallel_scheduler_backend` corresponding to different iterations.

   ```
   std::optional<P> receiver::try_query() noexcept;
   ```

2. *Returns*: an object that contains the property of type `P` if the receiver's environment has such a property, or an empty `optional` otherwise.

   - If the type erased receiver represented by this object has an environment for which `get_stop_token` returns an `inplace_stop_token` object, then `try_query<inplace_stop_token>()` returns an `optional` of a value equal with the stop token from the environment;
   - It is implementation defined for which other properties `P` the method `try_query` returns non-empty optional objects.

33.N.3 Class `parallel_scheduler_backend` [exec.sysctxrepl.psb]

1. `parallel_scheduler_backend` encapsulates the implementation details of the parallel scheduler.

```
virtual void parallel_scheduler_backend::schedule(receiver& r,
std::span<std::byte> s) noexcept = 0;
```

2. *Effects*:

   - schedules new work on a thread belonging to the execution context represented by `this`;
   - eventually, one of the following methods is called on the given object `r`:
     - `r.set_value()`, if no error occurs, and the work is not cancelled;
     - `r.set_error(e)`, if an error occurs, where `e` is an object of the `exception_ptr` type representing an error;
     - `r.set_stopped()`, if the work is cancelled.

3. *Remarks*:

   - The caller must guarantees that, until one of `r.set_value()`, `r.set_error()` or `r.set_stopped()` is called, the following holds:
     - `this` object of the `parallel_scheduler_backend` instance is valid;
     - `r` and `s` are valid;
     - if `s.size() > 0` is `true`, the memory pointed by `s.data()` is valid.

```
virtual void
parallel_scheduler_backend::schedule_bulk_chunked(uint32_t n,
bulk_item_receiver& r, std::span<std::byte> s) noexcept = 0;
```

4. *Effects*: same as for `schedule(r, s)`, and in addition:

   - if `r.execute(b, e)` is called, then `0` <= `b` < `e` <= `n`;
   - all calls to `r.execute(b, e)` are done with disjoint intervals [`b`, `e`);
   - if `r.set_value()` is called, then for each `i` in [`0`, `n`), there is a call to `r.execute(b, e)` such as `b` <= `i` < `e`;
   - all calls to `r.execute()` happen before the call to either `r.set_value()`, `r.set_error()`, or `r.set_stopped()`;
   - all calls to `r.execute()` need to be made on threads that belong to the execution context represented by `this`.

5. *Remarks*: same as for `schedule(r, s)`.

```
virtual void
parallel_scheduler_backend::schedule_bulk_unchunked(uint32_t n,
bulk_item_receiver& r, std::span<std::byte> s) noexcept = 0;
```

6. *Effects*: same as for `schedule_bulk_chunked(n, r, s)`, with the difference that for all calls `r.execute(b, e)`, `e == b + 1`.
7. *Remarks*: same as for `schedule(r, s)`.

7. Polls

7.1. SG1, Wrocław, Poland, 2024

SG1 provided the following feedback, through a poll:

1. Forward P2079R5 to LEWG for C++26 with changes:
   - Remove the wording about delegatee schedulers
   - Add wording about set_error

SF	F	N	A	SA
3	2	1	0	0

Unanimous consent

7.2. LEWG, Wrocław, Poland, 2024

LEWG provided the following feedback, through polls:

1. POLL: We support mandating (by specifying in the standard) replaceability, as described in: “P2079R5 System execution context”

SF	F	N	A	SA
10	5	1	0	1

Attendance: 15 IP, 7 online

# of Authors: 2

Author's Position: 2x SF

Outcome: Consensus in favor

SA: I think using this central point is equivalent to using global and is bad software engineering practice and I wouldn't like to see it in the standard.

2. POLL: We support mandating the specific mechanism(s) (link time/runtime/both) of replaceability as described in: "P2079R5 System execution context"

SF	F	N	A	SA
5	6	3	0	2

Attendance: 15 IP, 7 online

# of Authors: 2

Author's Position: 1x SF 1x N

Outcome: Consensus in favor

SA: I don't believe it's actually possible to describe this in terms of the abstract machine, we've never done that.

3. POLL: We support specifying an API for the replaceability mechanism(s), as described in: "P2079R5 System execution context"

SF	F	N	A	SA
7	6	2	0	1

Attendance: 15 (IP), 7 (R)

# of Authors: 2

Author's Position: 2x SF

Outcome: Consensus in favor

4. POLL: For lifetime: "P2079R5 System execution context" (approval poll - allowed to vote multiple times):

    1. Not specified in the standard (valid to use the system scheduler everywhere) | 11 |

2. Allow lifetime to start before main() (allow use of scheduler before entering main but not after main returns) | 1 |
3. Constrain lifetime to only be inside of main() | 8 |

Attendance: 15 (IP), 7 (R)

Outcome: Slight preference towards 1 (If authors advocate for 3, they need to add rationale for it)

In addition to these polls, LEWG provided the following action items:

- Add noexcept to move, copy constructor, move/copy assignment operators (special member functions).
- Strike system_scheudler query(get_completion_scheduler_t) overload.
- Add Lvalue connectability.
- Add normative recommendation / non-normative note (if normative - should not be in a note) for sharability in implementations, to be decided later.

7.3. SG1, Hagenberg, Austria, 2025

SG1 provided the following feedback, through a forwarding poll:

7. Forward P2079R6 to LEWG with the following changes towards C++26:

   A. Move the name `get_system_scheduler` to a separate paper, we hope with a stronger progress guarantee.

   B. Rename the current `get_system_scheduler` to something with "parallel" in the name, like `get_parallel_scheduler`, and retain the property that it has the parallel progress guarantee.

   C. Remove run-time replaceability.

```
SF | F | N | A | SA
 5 | 5 | 0 | 1 | 0
```

   Consensus

   Against: LB: I’m a bit concerned about some of the lifetime implications of the API changes. I’d rather see it back in SG1 again before it goes to LEWG.

7.4. LEWG, Hagenberg, Austria, 2025

POLL: in P2079R7: rename `system_context_replaceability::parallel_scheduler` to `system_context_replaceability::parallel_scheduler_backend`

Outcome: No objection to UC (in favor)

POLL: in P2079R7: rename `system_context_replaceability::bulk_item_receiver::start(...)` to `system_context_replaceability::bulk_item_receiver::execute(...)`

Outcome: No objection to UC (in favor)

POLL: in P2079R7: replace `system_context_replaceability::storage` with a `span`

```
SF | F | N | A | SA
 8 | 3 | 3 | 0 | 0
```

Attendance: 21 (IP) + 2 (R)

Author's Position: 2xN, WF

Outcome: Strong consensus in favour

POLL: Make the modifications above (rename, turn pointers into refs, replace storage with span) and forward P2079R7 to LWG for C++26.

```
SF | F | N | A | SA
 6 | 5 | 2 | 1 | 1
```

Attendance: 21 (IP) + 2 (R)

Author's Position: SF, F, N

Outcome: Consensus in favour

§ References

§ Informative References

[EXEC]

ISO WG21. *Working Draft: Programming Languages — C++ -- Execution control library*. URL: https://eel.is/c++draft/exec

[P0443R14]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, H. Carter Edwards, Gordon Brown, D. S. Hollman. *A Unified Executors Proposal for C++*. 15 September 2020. URL: https://wg21.link/p0443r14

[P2079R1]

Ruslan Arutyunyan, Michael Voss. *Parallel Executor*. 15 August 2020. URL: https://wg21.link/p2079r1

[P2079R2]

Lee Howes, Ruslan Arutyunyan, Michael Voss. *System execution context*. 15 January 2022. URL: https://wg21.link/p2079r2

[P2300R10]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 28 June 2024. URL: https://wg21.link/p2300r10

[P2900R8]

Joshua Berne, Timur Doumler, Andrzej Krzemieński. *Contracts for C++*. 27 July 2024. URL: https://wg21.link/p2900r8

[P3109R0]

Lewis Baker, Eric Niebler, Kirk Shoop, Lucian Radu. *A plan for std::execution for C++26*. 12 February 2024. URL: https://wg21.link/p3109r0

[P3481R1]

Lucian Radu Teodorescu; Lewis Baker; Ruslan Arutyunyan. *std::execution::bulk() issues*. URL: https://wg21.link/P3481R1